SOLE ATELIER: how the pieces fit

A study notebook for the project. Every line carries an evidence label so the page never claims more than the evidence supports.

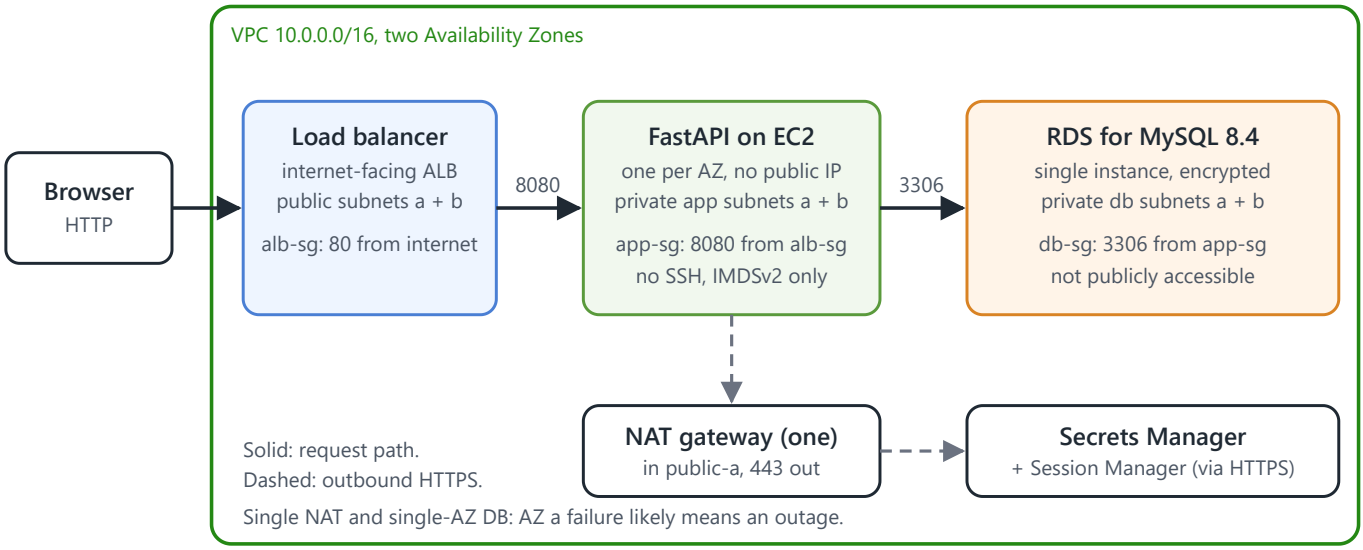
VERIFIED	REPORTED	CONCEPTUAL	UNVERIFIED	PLANNED
Run or seen in this project's recorded evidence	In the author's notes, no result captured	A general fact, not run here	Believed, not evidenced	An intention, not an event

1 What this project teaches

- VERIFIED How an application enforces prices, stock and validation on the server, and how to prove it with tests that fail when the code is broken (91 tests, 6/6 deliberate bugs caught).
- PLANNED How to build a three-tier network by hand: public, private and database subnets, security-group chaining, NAT, ALB, RDS, IAM role, Session Manager.
- PLANNED How to prove an AWS claim: a configuration screenshot versus a behaviour test (for example, a failed connection to the database from outside).

Architecture (design, not deployed)

DESIGN ONLY. Nothing here is deployed.



PLANNED Target design. No AWS resource exists. Port numbers are destination ports.

Request flow

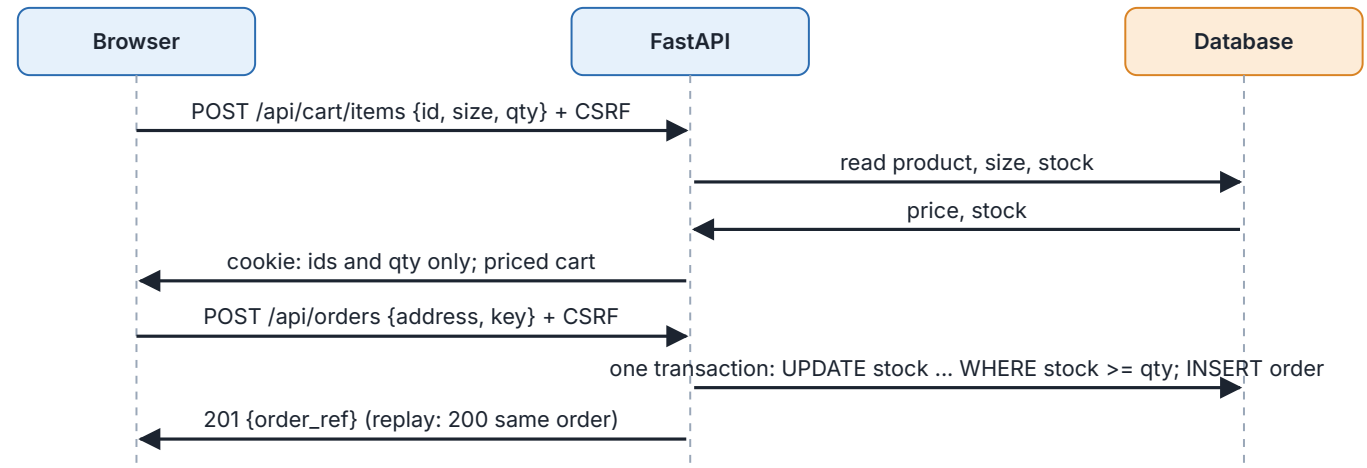
#	STEP	CONTROL	LABEL
1	Browser sends HTTP 80 to the internet-facing ALB	alb-sg	PLANNED
2	ALB forwards to FastAPI on port 8080 in a private subnet	app-sg from alb-sg	PLANNED
3	App validates input, reads and writes MySQL on 3306	db-sg from app-sg	PLANNED (code VERIFIED on SQLite)
4	At startup the app reads two secrets using its IAM role, through NAT on 443	role limited to two ARNs	PLANNED
5	ALB health check calls `/health`, which does not query the database	liveness only	VERIFIED locally

Why it matters: a healthy target group proves only that the ALB reached port 8080 and got a 200. It does not prove the database is secure or that checkout works.

Source: this repository only. Nothing here is a claimed deployment or metric.

The application: cart, orders and data

Cart and order workflow



VERIFIED Behaviour covered by automated tests on SQLite. Not run against MySQL or AWS.

The atomic stock reservation (from `backend/services/orders.py`)

```
UPDATE product_sizes SET stock = stock - :qty
 WHERE product_id = :p AND size = :s AND stock >= :qty
-- rowcount != 1  =>  409 "just sold out", roll back every line
```

Data model

TABLE	KEY COLUMNS	WHY
`products`	slug, name, price NUMERIC(10,2), category, badge, is_active	Money is decimal, never float
`product_sizes`	(product_id, size) key, stock ≥ 0	Stock is per size, which is what is actually bought
`orders`	public_id, idempotency_key (unique), totals, address, status	Unguessable reference; replay-safe
`order_items`	product_name, size, qty, unit_price, line_total	Snapshots: later price or name changes do not rewrite history

Why these choices (and the cost of each)

CHOICE	BENEFIT	COST OR LIMIT
Cart in a signed cookie	No table; works across two instances with a shared key	Per browser; lost when cookies clear
Idempotency key	Double click cannot create two orders	Needs a client-generated key
Liveness vs readiness checks	A database blip does not empty the load balancer	Instances can be “healthy” yet return 503
Strict CSP, no inline script	Limits XSS impact	All JS and CSS must be external files
SQLite locally, MySQL on RDS	Run and test with no AWS cost	MySQL behaviour unobserved until the AWS build

Source: backend/, frontend/, tests/, docs/architecture.md.

Network, security, commands and revision

Subnets

NAME	CIDR	INTERNET ROUTE
public-a / b	10.0.0.0/24, 10.0.1.0/24	IGW
app-a / b	10.0.10.0/24, 10.0.11.0/24	NAT (CP4)
db-a / b	10.0.20.0/24, 10.0.21.0/24	none

PLANNED A subnet is public only because its route table points at an internet gateway.

Security-group chain

GROUP	IN	OUT
alb-sg	80 from internet	8080 to app-sg
app-sg	8080 from alb-sg	3306 to db-sg, 443
db-sg	3306 from app-sg	none

PLANNED No rule for port 22 anywhere.

Commands

```
# local, run in this project
pytest                                  # 91 passed
python tools/mutation_check.py          # 6/6 caught
node tools/check-docs.mjs               # documentation rules
```

```
# conceptual: not run, needs the AWS build
python -m backend.init_db --admin-user admin --seed     # create tables once (admin, prompted password)
nc -vz <rds-endpoint> 3306                            # from outside the VPC: must time out
curl -s http://127.0.0.1:8080/health/ready            # on an instance: database ok
```

VERIFIED for the first block. CONCEPTUAL for the second; none has been run against AWS.

Validation checklist for the AWS build

TEST	PROVES	DOES NOT PROVE	STATUS
Failed connection to RDS from the internet (E-007)	Not reachable from outside	Not reachable from inside	PLANNED
Order row in RDS after an order via the ALB (E-008)	Browser to ALB to app to DB works	Behaviour under load	PLANNED
Two healthy targets in two AZs (E-009)	ALB reaches /health	DB security, checkout	PLANNED
Terminate one instance (E-012)	Service survives one instance	AZ-level failure	PLANNED

Revision questions

- What exactly makes a subnet public, and what is in the database subnets' route table?
- Why does `db-sg` name `app-sg` instead of a CIDR range?
- Why must a DB subnet group span two AZs when the instance runs in one?
- What would break if `/health` queried the database?
- Which parts of this project are verified, and which only designed?

Remember

Configuration is not behaviour. Say "planned" until I have my own screenshot and my own test.

Source: this repository only. Cost and price figures are intentionally absent.